

REQUIREMENT ANALYSIS

AGESTA – AI-Generated Smart Tax Assistant (India-Friendly)

1. User Roles Defined (WHO uses the system)

Why define user roles first?

- Functional requirements are written per role
- Use case diagram actors come directly from this list

1.1 Taxpayer (End User)

A taxpayer is an individual who needs to file income tax returns and compare tax regimes.

Responsibilities / Expectations:

- Enter personal income and expense details
- View tax calculations for both old and new regimes
- Receive AI-based recommendation on the better regime
- Understand tax comparison in simple language
- Access secure tax summary stored using blockchain

1.2 Admin (System Administrator)

An admin manages the system and ensures secure operation.

Responsibilities / Expectations:

- Manage user accounts and verify credentials
- Monitor tax comparison records
- Ensure secure blockchain storage of tax data
- Generate system usage reports

Academic Note: These roles will become Actors in the use case diagram.

2. Functional Requirements (WHAT the system SHALL do)

Definition:

Functional requirements describe the specific actions and services the system must provide to each user role.

Writing Rule (Very Important)

All functional requirements should be written as: "The system shall ..."

2.1 Taxpayer – Functional Requirements

1. The system shall allow taxpayers to register with basic personal details (Name, Email, PAN, AY).
2. The system shall allow taxpayers to log in using valid email and password.
3. The system shall allow taxpayers to enter gross income details (Salary, Interest, Dividends, etc.).
4. The system shall allow taxpayers to enter expense/deduction details (80C, 80D, HRA, etc.).

5. The system shall calculate total taxable income under the Old Tax Regime.
6. The system shall calculate total taxable income under the New Tax Regime.
7. The system shall calculate tax liability for both regimes using applicable tax slabs.
8. The system shall compare both regimes and highlight the financially beneficial option.
9. The system shall provide AI-based recommendation with reasoning for the better regime.
10. The system shall display tax savings or additional burden if switching regimes.
11. The system shall generate a clear, formatted tax comparison report.
12. The system shall allow taxpayers to download the tax summary as a PDF.
13. The system shall store the tax summary securely using blockchain technology.
14. The system shall allow taxpayers to retrieve previously stored tax summaries.

2.2 Admin – Functional Requirements

15. The system shall allow admin to log in securely with admin credentials.
16. The system shall allow admin to view all registered taxpayer accounts.
17. The system shall allow admin to activate or deactivate user accounts.
18. The system shall allow admin to view all tax comparison records.
19. The system shall allow admin to view blockchain-stored tax summaries.
20. The system shall allow admin to verify data integrity using blockchain records.
21. The system shall generate system usage and audit reports.

2.3 Functional Requirements → Use Case Mapping

Functional Requirement	Use Case Name
Enter income details	Provide Financial Details
Compare regimes	Compare Tax Regimes
Receive recommendation	Get AI Recommendation
Store tax summary	Store Blockchain Record
Monitor users	Admin Dashboard

Note: Each functional requirement becomes a use case and will be represented as an ellipse in the use case diagram.

3. Non-Functional Requirements (HOW WELL the system should work)

Definition:

Non-functional requirements define quality attributes such as performance, security, and usability, rather than specific actions.

3.1 Performance

- The system shall respond to user input within 2–3 seconds.
- Tax calculations and regime comparisons shall complete within 5 seconds.

3.2 Security

- The system shall authenticate users before allowing access.
- Passwords shall be encrypted using industry-standard hashing (e.g., bcrypt).
- Users shall access only features and data permitted to their role.
- Tax summaries shall be encrypted before blockchain storage.
- The system shall maintain audit logs for all user actions.

3.3 Data Integrity

- Tax records stored on blockchain shall be immutable and tamper-proof.
- Blockchain records shall include cryptographic hashes for verification.

3.4 Usability

- The interface shall be simple and intuitive for non-technical users.
- Tax explanations shall use simple language, avoiding jargon.
- The system shall support multiple languages (Hindi, English).

3.5 Reliability

- The system shall prevent duplicate tax calculations for the same user in the same session.
- Blockchain transactions shall be confirmed before marking records as stored.
- The system shall achieve 99% uptime.

3.6 Compatibility

- The system shall be accessible through modern web browsers (Chrome, Firefox, Safari, Edge).
- The system shall support responsive design for desktop and mobile devices.

3.7 Scalability

- The system shall support multiple concurrent users without performance degradation.

3.8 Non-Functional Requirements Summary Table

Category	Requirement
Performance	2–3 seconds response time; 5 seconds for calculations
Security	Authentication, encryption, role-based access, audit logs
Data Integrity	Immutable blockchain storage with cryptographic verification
Usability	Simple UI, clear explanations, multi-language support
Reliability	99% uptime, prevent duplicates, confirm transactions
Compatibility	Modern browsers, responsive design for mobile
Scalability	Support multiple concurrent users

4. Key Takeaways

Roadmap to Next Phase:

- Requirements Analysis converts the problem statement into clear system actions and quality expectations.
- User Roles guide the creation of actors in use case diagrams.
- Functional Requirements become use cases.
- Non-Functional Requirements ensure quality and security.